

Week 5 Skill

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
divya@Divyasree:~$ mkdir directory_project
divya@Divyasree:~$ cd directory_project
divya@Divyasree:~/directory_project$ pwd
/home/divya/directory_project
divya@Divyasree:~/directory_project$ nano directory_navigation.c
divya@Divyasree:~/directory_project$ ls
directory_navigation.c
divya@Divyasree:~/directory_project$ gcc directory_navigation.c -o directory_navigation
divya@Divyasree:~/directory_project$ la
directory_navigation  directory_navigation.c
divya@Divyasree:~/directory_project$ ./directory_navigation
=== Directory Navigation Program ===
Current Directory: /home/divya/directory_project

Commands: pwd | cd <path> | cd - | exit
>> pwd
Current Directory: /home/divya/directory_project

Commands: pwd | cd <path> | cd - | exit
>> cd /tmp
Directory changed successfully!
Previous Directory: /home/divya/directory_project
Current Directory: /tmp

Commands: pwd | cd <path> | cd - | exit
>> pwd
Current Directory: /tmp

Commands: pwd | cd <path> | cd - | exit
>> cd -
Changed to: /home/divya/directory_project

Commands: pwd | cd <path> | cd - | exit
```

```
Current Directory: /home/divya/directory_project

Commands: pwd | cd <path> | cd - | exit
>> cd /tmp
Directory changed successfully!
Previous Directory: /home/divya/directory_project
Current Directory: /tmp

Commands: pwd | cd <path> | cd - | exit
>> pwd
Current Directory: /tmp

Commands: pwd | cd <path> | cd - | exit
>> cd -
Changed to: /home/divya/directory_project

Commands: pwd | cd <path> | cd - | exit
>> cd abc
Error: No such file or directory

Commands: pwd | cd <path> | cd - | exit
>> cd /
Directory changed successfully!
Previous Directory: /home/divya/directory_project
Current Directory: /

Commands: pwd | cd <path> | cd - | exit
>> cd ..
Directory changed successfully!
Previous Directory: /
Current Directory: /

Commands: pwd | cd <path> | cd - | exit
```

```

Commands: pwd | cd <path> | cd - | exit
>> pwd
Current Directory: /

Commands: pwd | cd <path> | cd - | exit
>> exit
Exiting program...
divya@Divyasree:~/directory_project$ |

```

Code:

```

GNU nano 7.2 directory_navigation.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

int main() {
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";
    char path[PATH_MAX];

    printf("=== Directory Navigation Program ===\n");

    // Get initial working directory
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return 1;
    }

    printf("Current Directory: %s\n", current_dir);

    while (1) {
        printf("\nCommands: pwd | cd <path> | cd - | exit\n");
        printf(">> ");

        fgets(path, sizeof(path), stdin);
        path[strcspn(path, "\n")] = '\0';

        // Exit
        printf("Exiting program...\n");
        break;

        // pwd command
        if (strcmp(path, "pwd") == 0) {
            if (getcwd(current_dir, sizeof(current_dir)) != NULL)
                printf("Current Directory: %s\n", current_dir);
            else
                perror("getcwd");

            continue;
        }

        // cd - command
        if (strcmp(path, "cd -") == 0) {
            if (strlen(previous_dir) == 0) {
                printf("No previous directory available.\n");
                continue;
            }

            char temp[PATH_MAX];
            strcpy(temp, current_dir);

            if (chdir(previous_dir) == -1) {
                perror("chdir");
                continue;
            }
        }
    }
}

```

```

GNU nano 7.2 directory_navigation.c

    strcpy(previous_dir, temp);

    getcwd(current_dir, sizeof(current_dir));

    printf("Changed to: %s\n", current_dir);

    continue;
}

// cd <path> command
if (strncmp(path, "cd ", 3) == 0) {

    char *new_dir = path + 3;

    if (strlen(new_dir) == 0) {
        printf("Error: Directory path cannot be empty.\n");
        continue;
    }

    // Store current directory
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        continue;
    }

    // Try changing directory
    if (chdir(new_dir) == -1) {
        perror("Error");
    }

    if (chdir(new_dir) == -1) {
        perror("Error");
        continue;
    }

    // Store previous directory
    strcpy(previous_dir, current_dir);

    // Get new current directory
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        continue;
    }

    printf("Directory changed successfully!\n");
    printf("Previous Directory: %s\n", previous_dir);
    printf("Current Directory: %s\n", current_dir);

    continue;
}

printf("Invalid command.\n");
}

return 0;
}

```

TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

divya@Divyasree:~$ cd ~/directory_project
divya@Divyasree:~/directory_project$ pwd
/home/divya/directory_project
divya@Divyasree:~/directory_project$ ls
directory_navigation  directory_navigation.c
divya@Divyasree:~/directory_project$ builtin_dispatch.c
builtin_dispatch.c: command not found
divya@Divyasree:~/directory_project$ nano builtin_dispatch.c
divya@Divyasree:~/directory_project$ ls
builtin_dispatch.c  directory_navigation.c
directory_navigation
divya@Divyasree:~/directory_project$ gcc builtin_dispatch.c -o builtin_dispatch
builtin_dispatch.c:7: warning: "MAX_INPUT" redefined
   7 | #define MAX_INPUT 256
     |
In file included from /usr/include/x86_64-linux-gnu/bits/local_lim.h:38,
                  from /usr/include/x86_64-linux-gnu/bits/posix1_lim.h:161,
                  from /usr/include/limits.h:195,
                  from /usr/lib/gcc/x86_64-linux-gnu/13/include/limits.h:205,
                  from /usr/lib/gcc/x86_64-linux-gnu/13/include/syslimits.h:7,
                  from /usr/lib/gcc/x86_64-linux-gnu/13/include/limits.h:34,
                  from builtin_dispatch.c:5:
/usr/include/linux/limits.h:11: note: this is the location of the previous definition
   11 | #define MAX_INPUT      255      /* size of the type-ahead buffer */

```

```

divya@Divyasree:~/directory_project$ ls
builtin_dispatch  directory_navigation
builtin_dispatch.c  directory_navigation.c
divya@Divyasree:~/directory_project$ ./builtin_dispatch
=== Built-in Command Dispatch Shell ===
Type 'help' to view commands.

```

```

mini-shell> help

```

```

Available Built-in Commands:
pwd          - Display current directory
cd <path>    - Change directory
help        - Display available commands
exit        - Exit the shell

```

```

mini-shell> pwd
Current Directory: /home/divya/directory_project
mini-shell> cd /tmp
Directory changed successfully.
Current Directory: /tmp
mini-shell> pwd
Current Directory: /tmp
mini-shell> hello
Invalid command: hello
Type 'help' to see available commands.
mini-shell> cd abc
cd: No such file or directory
mini-shell>

```

```

GNU nano 7.2                               builtin_dispatch.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_INPUT 256

// Built-in command: pwd
void builtin_pwd() {
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current Directory: %s\n", cwd);
    else
        perror("pwd");
}

// Built-in command: help
void builtin_help() {
    printf("\nAvailable Built-in Commands:\n");
    printf("pwd      - Display current directory\n");
    printf("cd <path> - Change directory\n");
    printf("help     - Display available commands\n");
    printf("exit     - Exit the shell\n");
}

// Built-in command: cd
void builtin_cd(char *path) {

```

```

    if (path == NULL || strlen(path) == 0) {
        printf("Error: cd requires a directory path.\n");
        return;
    }

    if (chdir(path) == -1) {
        perror("cd");
        return;
    }

    printf("Directory changed successfully.\n");
    builtin_pwd();
}

// Dispatch table
struct builtin {
    char *name;
    void (*function)(char *);
};

// Execute command
int dispatch_command(char *input) {

    if (strcmp(input, "pwd") == 0) {
        builtin_pwd();
        return 1;
    }

    if (strcmp(input, "help") == 0) {

```

```

    if (strcmp(input, "help") == 0) {
        builtin_help();
        return 1;
    }

    if (strncmp(input, "cd ", 3) == 0) {
        builtin_cd(input + 3);
        return 1;
    }

    if (strcmp(input, "cd") == 0) {
        printf("Error: cd requires a directory path.\n");
        return 1;
    }

    if (strcmp(input, "exit") == 0) {
        printf("Exiting Mini Shell...\n");
        return 0;
    }

    printf("Invalid command: %s\n", input);
    printf("Type 'help' to see available commands.\n");

    return 1;
}

int main() {

int main() {

    char input[MAX_INPUT];
    int running = 1;

    printf("=== Built-in Command Dispatch Shell ===\n");
    printf("Type 'help' to view commands.\n\n");

    while (running) {

        printf("mini-shell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        running = dispatch_command(input);
    }

    return 0;
}

```

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir directory_state
divya@Divyasree:~$ cd directory_state
divya@Divyasree:~/directory_state$ pwd
/home/divya/directory_state
divya@Divyasree:~/directory_state$ nano directory_manager.c
divya@Divyasree:~/directory_state$ gcc directory_manager.c -o directory_manager
divya@Divyasree:~/directory_state$ ls
directory_manager  directory_manager.c
divya@Divyasree:~/directory_state$ ./directory_manager
===== Directory Management System =====
Current Directory: /home/divya/directory_state
Current state saved successfully.

Enter command (pwd / exit): pwd
Current Directory: /home/divya/directory_state

Enter command (pwd / exit): hello
Invalid command. Use 'pwd' or 'exit'.

Enter command (pwd / exit): exit
Exit request received.
Current state saved successfully.
Program terminating...

Cleaning up resources...
Resources released successfully.
divya@Divyasree:~/directory_state$ cat state.txt
/home/divya/directory_state
divya@Divyasree:~/directory_state$ pwd
/home/divya/directory_state
divya@Divyasree:~/directory_state$ ls -l
total 28
-rwxr-xr-x 1 divya divya 16528 Sep 27 03:51 directory_manager
-rw-r--r-- 1 divya divya 1531 Sep 27 03:50 directory_manager.c
-rw-r--r-- 1 divya divya 28 Sep 27 03:55 state.txt

```

Code:

```

GNU nano 7.2 directory_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

void cleanup() {
    printf("\nCleaning up resources...\n");
    printf("Resources released successfully.\n");
}

void save_state(const char *path) {
    FILE *file = fopen("state.txt", "w");

    if (file == NULL) {
        perror("Unable to save state");
        return;
    }

    fprintf(file, "%s\n", path);
    fclose(file);

    printf("Current state saved successfully.\n");
}

int main() {
    char current_path[PATH_MAX];
    char command[100];

```

```

GNU nano 7.2 directory_manager.c
int main() {
    char current_path[PATH_MAX];
    char command[100];

    atexit(cleanup);

    printf("==== Directory Management System ==== \n");

    if (getcwd(current_path, sizeof(current_path)) != NULL) {
        printf("Current Directory: %s \n", current_path);
    } else {
        perror("getcwd");
        return 1;
    }

    save_state(current_path);

    while (1) {
        printf("\nEnter command (pwd / exit): ");
        scanf("%99s", command);

        if (strcmp(command, "pwd") == 0) {
            if (getcwd(current_path, sizeof(current_path)) != NULL) {
                printf("Current Directory: %s \n", current_path);
            } else {
                perror("getcwd");
            }
        }
        else if (strcmp(command, "exit") == 0) {
            printf("Exit request received. \n");
            save_state(current_path);
            printf("Program terminating... \n");
            exit(0);
        }
        else {
            printf("Invalid command. Use 'pwd' or 'exit'. \n");
        }
    }

    return 0;
}

```

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```

divya@Divyasree:~$ mkdir export_project
divya@Divyasree:~$ cd export_project
divya@Divyasree:~/export_project$ pwd
/home/divya/export_project
divya@Divyasree:~/export_project$ nano export_manager.c
divya@Divyasree:~/export_project$ gcc export_manager.c -o export_manager
divya@Divyasree:~/export_project$ ls
export_manager  export_manager.c
divya@Divyasree:~/export_project$ ./export_manager
===== Export Environment Manager =====

Enter command:
export NAME=value
test NAME
exit
> export NAME=Divya
Variable 'NAME' already exists.
Old value: Divyasree
Variable updated successfully.
NAME=Divya

Enter command:
export NAME=value
test NAME
exit
> test NAME

--- Child Process ---
Child received NAME=Divya
--- Child process completed ---

Enter command:
export NAME=value

```

```

--- Child Process ---
Child received NAME=Divya
--- Child process completed ---

Enter command:
export NAME=value
test NAME
exit
> export 123NAME=Hello
Error: Invalid variable name: 123NAME

Enter command:
export NAME=value
test NAME
exit
> export NAME-TEST=Hello
Error: Invalid variable name: NAME-TEST

Enter command:
export NAME=value
test NAME
exit
> exit
Unknown command.

Enter command:
export NAME=value
test NAME
exit
> exit
Exiting program...
divya@Divyasree:~/export_project$ |

```

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200

// Validate environment variable name
int valid_name(const char *name)
{
    if (name == NULL || name[0] == '\0')
        return 0;

    // First character must be letter or underscore
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    // Remaining characters can be letters, digits, or underscore
    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

```

```

GNU nano 7.2 export_manager.c
void process_export(char *input)
{
    char *command = strtok(input, " \t");

    if (command == NULL || strcmp(command, "export") != 0)
    {
        printf("Invalid command.\n");
        return;
    }

    char *assignment = strtok(NULL, " \t");

    if (assignment == NULL)
    {
        printf("Usage: export NAME=value\n");
        return;
    }

    // Find '='
    char *equal = strchr(assignment, '=');

    if (equal == NULL)
    {
        printf("Error: export must use NAME=value format.\n");
        return;
    }

    // Separate name and value
    *equal = '\0';
}

```

```

GNU nano 7.2 export_manager.c
    return;
}

// Check whether variable already exists
char *old_value = getenv(name);

if (old_value != NULL)
{
    printf("Variable '%s' already exists.\n", name);
    printf("Old value: %s\n", old_value);
}
else
{
    printf("Creating new variable '%s'.\n", name);
}

// Update environment variable
if (setenv(name, value, 1) == 0)
{
    printf("Variable updated successfully.\n");
    printf("%s=%s\n", name, getenv(name));
}
else
{
    perror("setenv");
}
}

// Test whether child process receives variable

```

```

GNU nano 7.2 export_manager.c
void test_child_process(const char *name)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        printf("\n--- Child Process ---\n");

        char *value = getenv(name);

        if (value != NULL)
            printf("Child received %s=%s\n", name, value);
        else
            printf("Child cannot find variable %s\n", name);

        exit(0);
    }
    else
    {
        wait(NULL);
        printf("--- Child process completed ---\n");
    }
}

```

```

GNU nano 7.2 export_manager.c
int main()
{
    char input[MAX_INPUT];

    printf("==== Export Environment Manager =====\n");

    while (1)
    {
        printf("\nEnter command:\n");
        printf("export NAME=value\n");
        printf("test NAME\n");
        printf("exit\n");
        printf("> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting program...\n");
            break;
        }

        if (strncmp(input, "export ", 7) == 0)
        {
            process_export(input);

```

```

            break;
        }

        if (strncmp(input, "export ", 7) == 0)
        {
            process_export(input);
        }
        else if (strncmp(input, "test ", 5) == 0)
        {
            char *name = input + 5;

            if (valid_name(name))
                test_child_process(name);
            else
                printf("Invalid variable name.\n");
        }
        else
        {
            printf("Unknown command.\n");
        }
    }

    return 0;
}

```